

Final Proje Raporu

EGO-Benzeri Sentetik Veri ile Gerçek Zamanlı Otobus Takip ve Yoğunluk Analiz Sistemi

Gerçek Zamanlı Bulut Tabanlı Uygulama Çalışması

Ders	BLM3522 - Bulut Bilişim Ve Uygulamaları
Öğrenci Numarası	21290560
Öğrenci Adı Soyadı	Ahmet Emre Özcan
GitHub Repo	https://github.com/eemrezcan/ego-like-realtime-bus-tracking

Ankara, 2026

Özet

Bu calismada, sentetik veri ureten bir toplu tasima telemetri sistemi gelistirilmis ve bu verinin MQTT tabanli bir akis ile AWS uzerinde gercek zamanli olarak islenmesi saglanmistir. Projede otobusler, belirli araliklarla konum, hiz, hat bilgisi ve kart basimina dayali binis sayisi ureten sanal cihazlar olarak modellenmistir. Uretilen telemetri verileri AWS IoT Core uzerinden sisteme alinmis, IoT kurali araciligiyla Amazon Kinesis Data Streams katmanina aktarilmis ve AWS Lambda fonksiyonu ile zenginlestirilerek Amazon DynamoDB tablolarina yazilmistir. Islenen veriler FastAPI tabanli bir okuma katmani ile sunulmus ve Streamlit tabanli bir dashboard uzerinden canli olarak gorsellestirilmistir.

Calismada ham veri ile turetilmis veri acik bicimde ayrilmistir. Konum, hiz ve binis sayisi simulator tarafinda uretilirken; tahmini varis suresi, tahmini inis miktarı, doluluk seviyesi ve gecikme bilgisi bulut tarafında hesaplanmistir. Bu yaklasim, projeyi yalnızca gorsel bir simulasyon olmaktan cikarmis ve gercek zamanli veri isleme mimarisinin uctan uca hayata gecirildigi bir uygulamaya donusturmuster. Elde edilen sonuc, AWS servisleri kullanilarak kurulan MQTT tabanli veri hattı ile sentetik otobus telemetrisinin basarili bicimde islenebildigini, depolanabildigini ve canli izleme amacli bir arayuze aktarilabildigini gostermistir.

Anahtar Kelimeler

MQTT, AWS IoT Core, Kinesis Data Streams, AWS Lambda, DynamoDB, FastAPI, Streamlit, real-time bus tracking

İçindekiler

1. Giriş
2. Problem Tanımı ve Kapsam
3. Sistem Mimarisi
4. Veri Modeli ve Olay Seması
5. Gerçekleme Ayrıntıları
6. AWS Dağıtımı ve Gerçek Ortam Doğrulaması
7. Sonuçlar ve Değerlendirme
8. Sonuç
9. Kaynakça

1. Giris

Gercek zamanli konum ve durum izleme sistemleri, modern toplu tasima altyapilarinin en gorunur bileşenlerinden biridir. Yolcularin otobusun mevcut konumunu, tahmini varis suresini ve hatta yogunluk seviyesini takip edebilmesi, hizmet kalitesini dogrudan etkileyen bir unsurdur. Benzer sekilde, isletmeciler kurumlar acisindan da canli telemetri verisi; hat optimizasyonu, gecikme tespiti ve operasyonel izleme icin yuksek deger tasir. Bu nedenle toplu tasimada gercek zamanli veri akisi, yalnızca bir bilgi ekrani problemi degil, ayni zamanda bulut tabanlı veri isleme ve servis butunlestirme problemidir.

Bu projede, gercek EGO verisi veya fiziksel cihazlar kullanilmadan, EGO-benzeri bir otobus takip sistemi sentetik veri ile modellenmistir. Buradaki temel amac, gercek bir kurum verisine bagli kalmadan, otobus telemetrisinin MQTT tabanlı bir yapı ile buluta aktarilmasi, bulutta islenmesi ve bir dashboard uzerinden canli olarak izlenmesi surecini gostermeektir. Bu tercih, dersin odak noktasini veri kaynagindan cok mimari tasarim, servis entegrasyonu ve gercek zamanli akis yonetimi uzerine tasimaktadir. Ayrica sentetik veri kullanimi, sistem tasarimini test etmeyi kolaylastirirken gercek kurum verisine bagimlilik gibi operasyonel kisitlari da ortadan kaldirmaktadir.

Calismanin ana katkisi, veri uretimi, bulut uzerinde zenginlestirme ve canli sunum katmanlarini tek bir yapı altinda birlestirmesidir. Simulator tarafinda yalnızca ham telemetri verisi uretilmis; ETA, tahmini inis miktarı, doluluk ve gecikme gibi karar verici bilgiler ise AWS uzerinde hesaplanmistir. Boylece sistem, sadece sabit veri gosteren bir demo olmaktan cikmis, gercek zamanli olay akisina dayali bulut tabanlı bir referans mimariye donusmustur.

2. Problem Tanimi ve Kapsam

Bu proje, EGO benzeri bir toplu tasima takip sisteminin teknik cekirdeginin sentetik veri ile modellenmesini hedeflemektedir. Temel problem, hareket halindeki otobuslerden gelen telemetri verisinin gercek zamanli olarak toplanmasi, bulut uzerinde islenmesi, operasyonel olarak anlamlı hale getirilmesi ve son kullanıcıya anlik olarak sunulmasıdır. Bu tanım, calismanin odagini basit bir harita gosteriminden ayirmakta; veri akisi, bulut servis entegrasyonu ve olay tabanlı isleme mantigini merkeze almaktadır.

Projede ham veri ile turetilen veri birbirinden acik bicimde ayrilmistir. Ham veri, simulator tarafinda uretilen bus_id, line_id, lat, lon, speed_kmh, next_stop_id, next_stop_name, boarding_count ve timestamp alanlarindan olusmaktadır. Buna karsilik estimated_eta_sec, estimated_alighting_count, estimated_occupancy_score, estimated_occupancy_level ve is_delayed alanlari bulut tarafinda hesaplanmistir. Bu ayrim, sistemin yalnızca veri gosteren bir simulasyon olmadigini, veriyi yorumlayan ve zenginlestiren bir bulut isleme katmani kurdugunu gostermektedir.

Calisma kapsaminda sistem, MVP mantigiyla sinirlendirilmis ve uc hat, hat basina birden fazla otobus ve tanimli durak/rota dizileri uzerinden ilerleyecek sekilde modellenmistir. Gercek EGO altyapisina baglanti kurulmamistir; fiziksel otobus cihazlari, GPS modulleri veya kart okuyucular kullanilmamistir. Makine ogrenmesi tabanlı tahminleme, kullanıcı girişi, mobil istemci, dinamik rota optimizasyonu ve gercek trafik verisi entegrasyonu gibi genisletilebilir bileşenler bu asamada bilerek kapsam disinda

birakilmistir. Boylece proje kapsamı, ders gereksinimlerine uygun olarak gerçek zamanlı veri akışı, bulut işleme, depolama ve görselleştirme ekseninde kontrollü biçimde tutulmuştur.

Bu kapsam sınırı, raporun değerlendirme mantığı açısından da önemlidir. Başarı ölçütü, gerçek bir belediye sisteminin tüm karmaşıklığını taklit etmek değil; seçilen mimarinin veri üretimi, MQTT tabanlı iletişim, AWS üzerinde zenginleştirme, DynamoDB ile depolama ve dashboard ile canlı izleme aşamalarını uçtan uca yerine getirebilmesidir.

3. Sistem Mimarisi

Genel veri akışı şu şekildedir:

Bus Simulator -> MQTT -> AWS IoT Core -> Kinesis -> Lambda -> DynamoDB -> FastAPI -> Streamlit

Bu mimaride simulator katmanı, otobusleri sanal veri üreticileri olarak modellemektedir. Her otobüs belirli bir hatta, yön bilgisine ve rota segmentine bağlı olarak konum güncellemekte; ardından bu veri MQTT topic'i üzerinden yayımlanmaktadır. MQTT'nin publish/subscribe yapısı, düşük ek yük ve cihaz-bulut haberleşmesine uygunluğu nedeniyle tercih edilmiştir. Bu seçim, otobüslerin birer IoT cihazı gibi davranmasını sağlayarak proje senaryosunu daha inandırıcı hale getirmektedir.

AWS IoT Core, MQTT istemcilerinden gelen mesajları güvenli biçimde kabul eden yönetilen giriş noktası olarak kullanılmıştır. IoT Core'a gelen mesajlar, bir IoT kuralı yardımıyla Amazon Kinesis Data Streams'e yönlendirilmektedir. Kinesis burada telemetri verisini sürekli ve düşük gecikmeli şekilde kabul eden akış omurgası olarak görev yapmaktadır. Bu seçim sayesinde veri girişi ile işleme katmanı birbirinden ayrılmıştır.

AWS Lambda, Kinesis üzerinden gelen olayları okuyup ETA, tahmini iniş, doluluk ve gecikme bilgisini hesaplamaktadır. Yani sistemin karar üreten kısmı bulut tarafında konumlanmaktadır. İşlenen veriler daha sonra biri anlık durum, diğeri ise telemetri geçmişi için kullanılan iki farklı DynamoDB tablosuna yazılmaktadır.

FastAPI veri sunum katmanını, Streamlit ise son kullanıcı arayüzünü sağlamaktadır. Bu yapı sayesinde veri kaynağı ile arayüz arasında net bir API katmanı konmuş, mimari hem daha modüler hem de daha savunulabilir hale getirilmiştir.

4. Veri Modeli ve Olay Seması

Sistem veri modeli üç ana bileşenden oluşmaktadır: hatlar, duraklar ve rotalar. data/ klasörü altında tanımlanan JSON dosyaları, simulatorun hangi otobüsün hangi hat üzerinde hareket edeceğini, hangi duraklara yaklaşacağını ve ETA hesaplamasında hangi segment bilgilerini kullanacağını belirlemektedir. Bu yapı, simulator ile bulut işleme katmanı arasında ortak bir referans modeli oluşturmaktadır. Başka bir ifadeyle veri modeli, yalnızca test verisi tutan statik bir klasör değil; hem simulatorun hareket mantığını hem de bulut tarafındaki hesaplama bağlamını belirleyen çekirdek tasarım katmanıdır.

Ham olay semasi, MQTT üzerinden gonderilen telemetri paketinin catisini tanımlamaktadır. Olay semasinin ayrı bir JSON Schema dosyasi ile tanımlanmasi, simulator, Lambda ve API katmanlari arasındaki veri sozlesmesini sabitlemiştir. Boylece bir katmanda yapılan degisiklik diger katmanlar için belirsizlik uretmemektedir.

Ham olay alanlari:

- event_id
- timestamp
- bus_id
- line_id
- route_direction
- lat
- lon
- speed_kmh
- next_stop_id
- next_stop_name
- boarding_count

Bulutta uretilen alanlar:

- estimated_eta_sec
- estimated_alighting_count
- estimated_occupancy_score
- estimated_occupancy_level
- is_delayed

Bu projedeki kritik tasarim karari, olculen veri ile tahmin edilen verinin ayrilmasidir. Binis sayisi simulator tarafında dogrudan uretilirken, inis miktarı bulutta tahmin edilmekte ve o ana kadarki doluluk skoru ile birlikte yeniden hesaplanmaktadır. ETA de yine konum, hiz ve durak mesafesi bilgisi üzerinden Lambda tarafında uretilmektedir. Bu tercih, sistemin basit bir sahte veri gosteriminden cikarak gercek zamanli veri zenginlestirme mimarisine donusmesini saglamistir. Ayni zamanda hangi bilginin gozlemlendigi hangi bilginin tahmin edildiği acik bicimde ayristirildiği için raporun teknik durustlugunu da guclendirmektedir.

5. Gercekleme Ayrıntilari

Simulator katmani Python ile gelistirilmis bir olay uretim aracidir. Bu katman, veri modelinde tanımlanan hat, durak ve rota bilgilerini okuyarak her hat için birden fazla otobus nesnesi uretmektedir. Her iterasyonda otobuslerin segment üzerindeki konumu guncellenmekte, hizlari yeniden hesaplanmakta ve

duraga bagli olarak binis sayisi uretilmektedir. Olusan payload, stdout, dosya veya MQTT modlari ile yayinlanabilmektedir. Projenin son asamasinda simulatora --continuous secenegi eklenmis ve Boylece dashboard tarafinda surekli veri akisi gozlenebilir hale gelmistir. AWS dogrulama asamasinda MQTT/TLS modu kullanilmis ve simulator AWS IoT Core endpoint'ine istemci sertifikasi ile baglanmistir.

Bulut isleme mantigi AWS Lambda uzerinde calisan Python kodu ile uygulanmistir. Lambda, Kinesis uzerinden gelen telemetry paketlerini alip once dogrulamakta, daha sonra ETA, tahmini inis, doluluk skoru ve gecikme bilgisini hesaplamaktadir. ETA hesabi, mevcut konum, hiz ve siradaki duraga ait segment bilgisi kullanilarak uretilmektedir. Tahmini inis miktarı ise mevcut doluluk ve durak baglamina dayali kural tabanli bir yaklasimla elde edilmektedir. Batch isleme mantiginda hata toleransi eklenmis; Boylece gecersiz bir kayit geldigi durumda tum batch yerine yalnızca hatali kayit atlanmistir. Bu karar, akis hattinin daha dayanikli calismasini saglamistir. Isleme kurallarının ana hedefi, ham veriyi olabildigince az varsayimla anlamlı operasyonel bilgiye donusturmektir.

DynamoDB tarafinda iki tablo kullanilmistir:

- bus_current_state
- telemetry_history

Bu iki tabloya ayrilmis tasarim, operasyonel izleme ile tarihsel kayit ihtiyacini birbirinden ayirmakta ve sistemin hem canli arayuz hem de raporlama acisinden daha kullanisli olmasini saglamaktadır.

FastAPI tarafinda temel endpoint'ler:

- /health
- /summary
- /buses
- /buses/{id}
- /lines

Bu yapi sayesinde veri sunumu katmani ile depolama katmani birbirinden ayrilmis, ayrıca yerel dosya modu ile DynamoDB modu arasinda gecis de kolaylastirilmistir. Boylece ayni dashboard hem lokal dogrulama asamasinda hem de AWS destekli gercek ortamda tekrar kullanilabilmistir.

Dashboard tarafinda sistem durumu banner'i, ozet metrik kartlari, hat bazli kartlar, otobus tablosu ve pydeck tabanli canli harita bulunmaktadır. Son asamada harita gorunurlugu artırilmis, otomatik yenileme davranisi eklenmis ve veri tazeligı gostergesi olusturulmustur. Bu tasarim karari, hem test edilebilirliği hem de demo sirasinda veri akisinin daha anlasilir sunulmasini kolaylastirmistir.

6. AWS Dagitimi ve Gercek Ortam Dogrulaması

AWS tarafinda erisim yonetimi icin IAM Identity Center tabanli SSO yapisi kullanilmistir. Kaynaklar eu-central-1 bolgesinde olusturulurken, SSO yapisinin us-east-1 tarafinda tanimli oldugu not edilmistir. Bu ayrim, dagitim surecinin ilk adimlarinda karsilasilan temel operasyonel noktalardan biri

olmuştur.

Oluşturulan temel kaynaklar:

- bus_current_state
- telemetry_history
- ego-bus-telemetry-stream
- ego-bus-lambda-role
- ego-bus-processor
- ego-bus-iot-rule-role
- ego_bus_telemetry_to_kinesis
- ego-bus-simulator-device

Doğrulama asama asama yapılmıştır:

1. Lambda doğrudan invoke edilerek test edildi
2. Kinesis üzerinden veri akışı kontrol edildi
3. AWS CLI ile IoT publish testi yapıldı
4. Python simulators MQTT/TLS ile AWS IoT Core'a bağlandı
5. FastAPI ve Streamlit katmanında son kullanıcı görünümü doğrulandı

Bu zincir, projenin yalnızca lokal ortamda değil, AWS üzerinde de uçtan uca çalıştığını göstermektedir. Doğrulama sürecinde yalnızca servislerin oluşması değil, gerçek veri geçişinin izlenmesi esas alınmıştır. Son aşamada FastAPI üzerinden okunan latest_timestamp bilgisinin değişmesi ve Streamlit dashboard'da otobüs konumlarının yenilenmesi, zincirin son kullanıcı tarafında da tamamlandığını göstermiştir.

7. Sonuçlar ve Değerlendirme

Sistemin temel hedefi yerine getirilmiştir. Sentetik otobüs telemetrisi MQTT ile buluta aktarılmış, AWS üzerinde işlenmiş, veritabanında saklanmış ve canlı dashboard üzerinde görüntülenmiştir. Yerel ve AWS destekli doğrulama adımlarında simulator, processor, API ve dashboard katmanlarının birlikte çalıştığı görülmüştür. Özellikle Simulator -> AWS IoT Core -> Kinesis -> Lambda -> DynamoDB -> FastAPI -> Streamlit zincirinin gerçek veri ile doğrulanması, çalışmanın teknik gücünü artıran en önemli sonuç olmuştur.

Doğrulanmış temel kazanımlar:

- MQTT tabanlı sentetik telemetri akışı kuruldu
- AWS IoT Core üzerinden gelen veri Kinesis'e aktarıldı
- Lambda ile zenginleştirme yapıldı

- Ham veri ile turetilmis veri ayrimi korundu
- DynamoDB uzerinden anlik durum okunabildi
- Dashboard canli veri akisina tepki verebilir hale geldi

Projede karsilasilan temel zorluklar daha cok entegrasyon katmaninda ortaya cikmistir. AWS CLI kimlik dogrulama yontemi, IoT sertifika dosyalarinin dogru konumlandirilmesi, Lambda paketleme sureci, batch icinde hatali kayitlarin etkisinin sinirlendirilmesi ve Streamlit tarafindaki canli yenileme davranisi bu surecte cozulmesi gereken basliklar olmustur. Ancak bu zorluklar ayni zamanda sistemin raporlanabilirliğini de guclendirmistir; cunku proje sadece teorik bir mimari olarak kalmamis, uygulama ve hata ayiklama asamalariyla birlikte gercek bir gelistirme surecine donusmustur.

Sistemin sinirlari da aciktir. Kullanilan veri tamamen sentetiktir; dolayisiyla gercek yolcu davranisi, trafik etkisi ve arac telemetrisi tam olarak modellenmemektedir. Yogunluk ve gecikme hesapları kural tabanlı olarak uretilmistir ve bu nedenle gercek dunyadaki karmasik davranisi tam olarak temsil etmemektedir. Buna ragmen dersin hedefleri acisindan sistem yeterli bir teknik derinlik sunmaktadir. Gelecek calismalarda makine ogrenmesi tabanlı ETA tahmini, gercek trafik verisi entegrasyonu, daha gelismis yolcu yogunluk modeli ve mobil istemci arayuzu gibi genisletmeler ele alinabilir.

8. Sonuc

Bu projede, EGO-benzeri bir otobus takip sisteminin sentetik veri ile modellenmesi ve bu verinin AWS tabanlı gercek zamanli bir akis mimarisi uzerinde islenmesi basarili bicimde gerceklestirilmistir. Sistem; veri uretimi, cihaz-bulut haberlesmesi, akisa dayali isleme, depolama ve gorsellestirme katmanlarini butunlesik bir sekilde bir araya getirmektedir. Ham veri ile turetilmis veri arasindaki ayrimin korunmasi ve karar ureten alanlarin bulutta hesaplanması, calismanin teknik anlamini guclendirmistir.

Sonuc olarak, bu calisma bulut bilisim dersinin temel hedefleriyle uyumlu bir sekilde, MQTT tabanlı gercek zamanli veri akisi ile AWS servis entegrasyonunu pratikte gosteren bir referans uygulama ortaya koymustur. Proje ayni zamanda, sentetik veri kullanilsa dahi dogru mimari kararlar, uygun servis secimi ve sistematik dogrulama adimlari ile akademik olarak savunulabilir, tekrar edilebilir ve gosterilebilir bir sistem kurulabilecegini gostermistir.

9. Kaynakca

Kaynakca ana dosyasi:

- references.bib